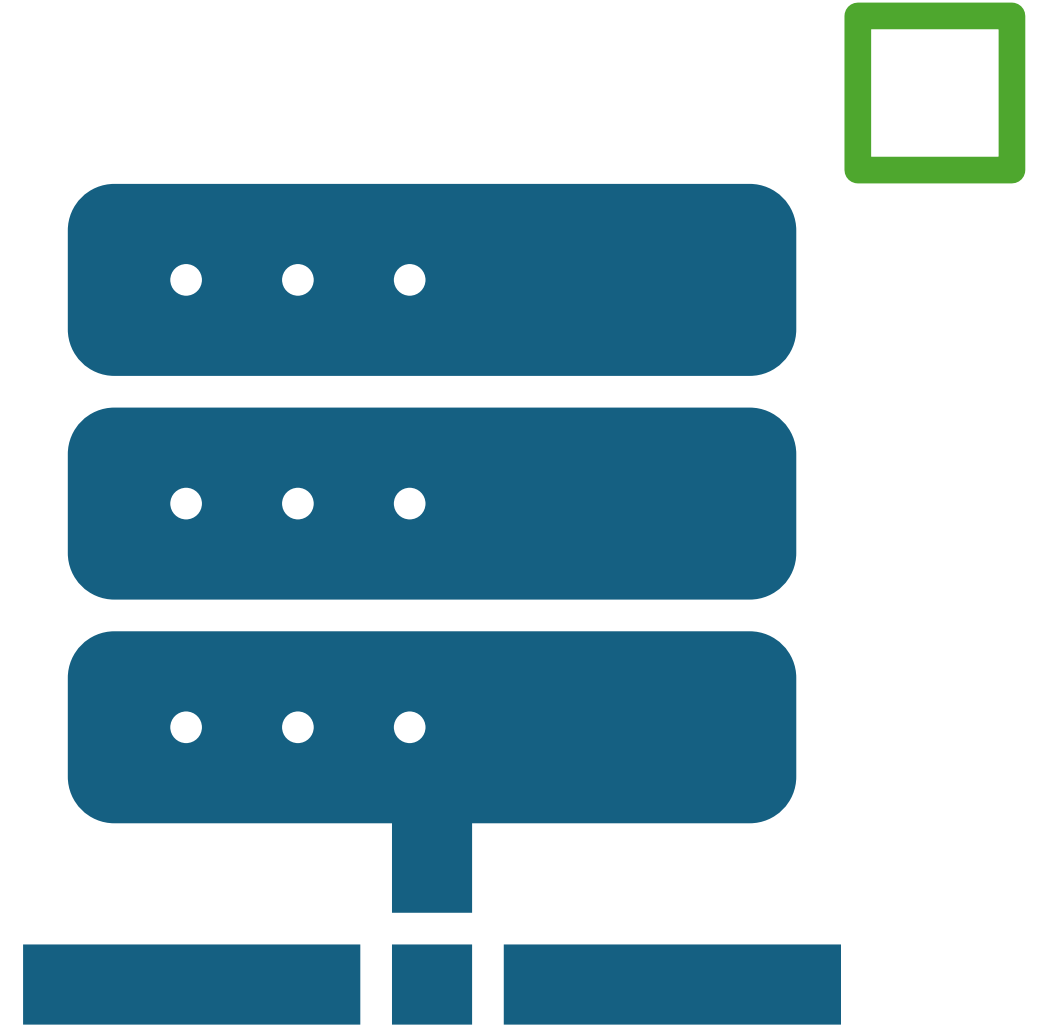


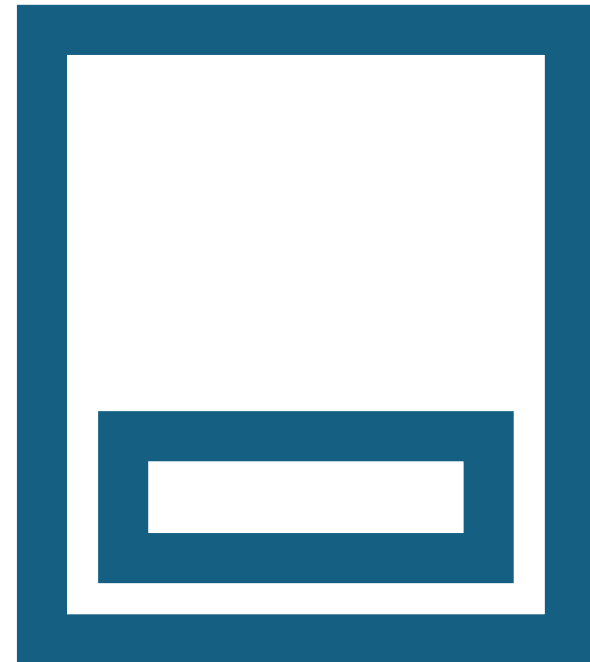
Stack Data Structure in C++ Array-Based Stack, Operations, Diagrams, Applications, and Live Code Demonstration

Presented by: Akash Patel
Instructor: Dr. Victor Govindaswamy



What Is a Stack?

- A **Stack** is a linear data structure where data is added and removed from only one end, called the **top**.
- We do not normally remove an item from the middle or bottom. The most recently added item stays at the top and is the first one available to remove.
- A Stack can store numbers, objects, function calls, browser pages, or other types of information depending on the program.
- The main idea is simple:
- **the newest item is handled first.**
- **Easy example:**
- Think about placing books one on top of another. If you want to remove a book safely, you normally remove the top book first.

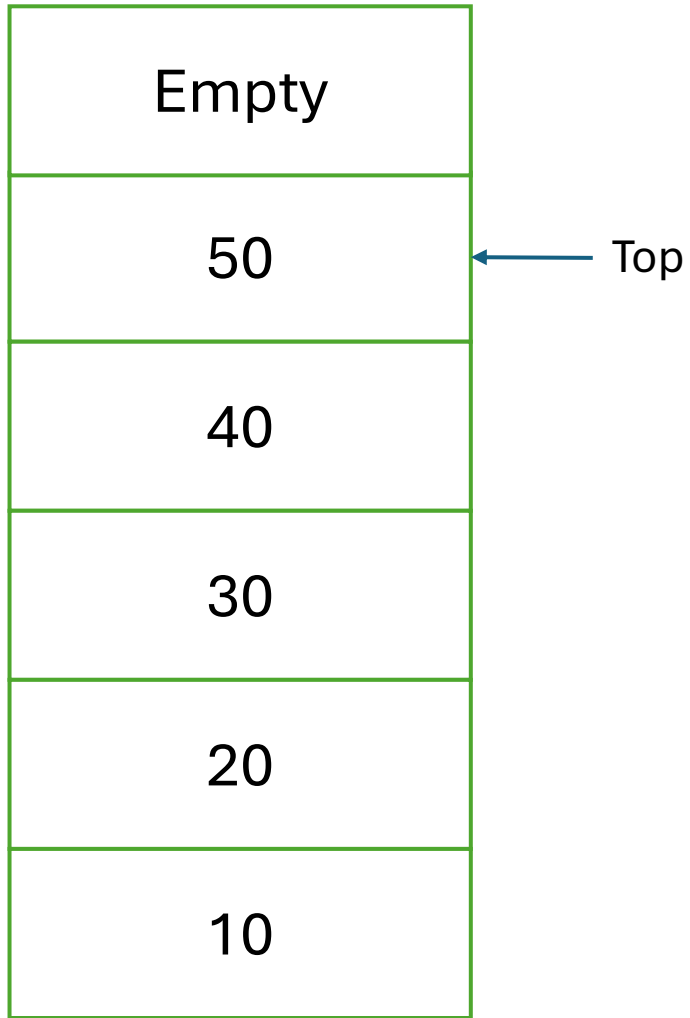


LIFO: Last In, First Out

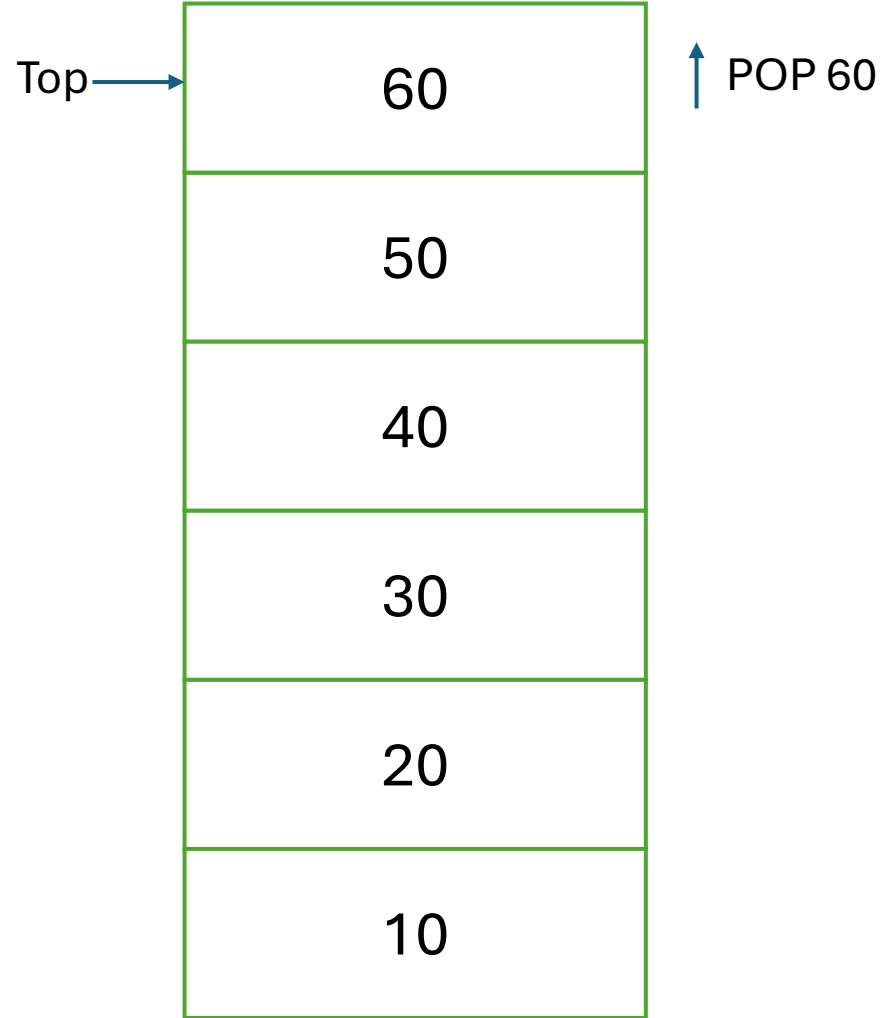
- **LIFO means “Last In, First Out.”**
- The **last value inserted** into the Stack becomes the **first value removed**.
- If we push 10, then 20, then 30, the top value becomes 30.
- When we perform a pop operation, 30 is removed first because it was the last value added.
- **Simple sequence:**
 - 10 → 20 → 30
 - Last value added = **30**
 - First value removed = **30**
- **Why this matters:**
 - LIFO gives the Stack a predictable order and makes it useful in situations where the most recent action needs to be handled first.

Stack – LIFO (Last In, First Out)

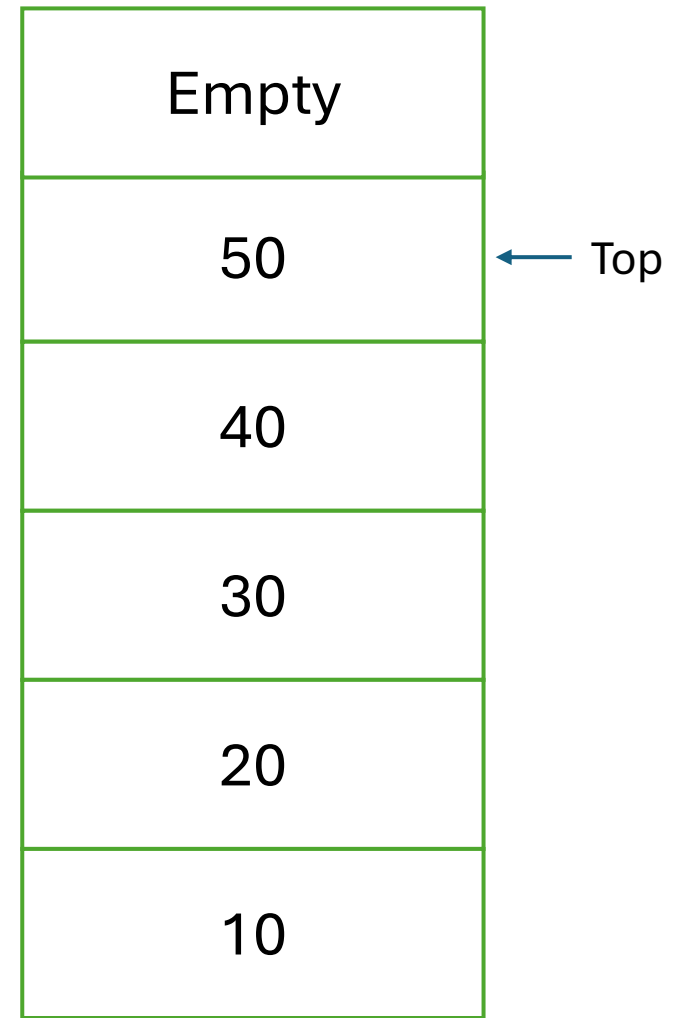
Before PUSH



After PUSH 60



After POP 60



Main Stack Operations

push(value)

Adds a new value to the **top** of the Stack.

Example: if the top value is 50 and we push 60, then 60 becomes the new top.

pop()

Removes the current value from the **top**.

If 60 is on top, pop() removes 60 and the previous value becomes the new top.

peek() / top()

Looks at the current top value **without removing it**.

This is useful when we only want to check what value will be removed next.

Checking and Displaying the Stack

➤ **isEmpty()**

- Checks whether the Stack contains any values.
- In my program, the Stack is empty when `top == -1`.
- This check prevents us from trying to remove data that does not exist.

➤ **isFull()**

- Checks whether the static array has reached its maximum capacity.
- In my code, the Stack is full when `top == SIZE - 1`.

➤ **displayAll()**

- Displays every stored value from **top to bottom**.
- It starts from the current top position and moves backward through the array.



How a Static Array Stores the Stack

- **My Stack is implemented using a static array.**
- A static array has a **fixed size decided before the program runs.**
- In my C++ program:
- `static const int SIZE = 5;`
- means the Stack can store a maximum of **5 values.**
- The array is declared as:
- `int arr[SIZ];`
- This is where the actual values are stored.
- The variable `top` keeps track of the current top position.
- When the Stack is first created:
- `top = -1;`
- This means there are no values in the Stack yet.
- **Important idea:**
- The array stores the data, while `top` tells the program which position is currently being used as the top of the Stack.

Stack Overflow and Stack Underflow

Stack Overflow

Stack overflow happens when we try to push another value into a Stack that is already full.

In a static-array Stack, there is no extra memory position available beyond the fixed size.

For example, if a Stack has capacity 5 and all 5 positions are filled, another push() should not be allowed.

Stack Underflow

Stack underflow happens when we try to remove a value from an empty Stack.

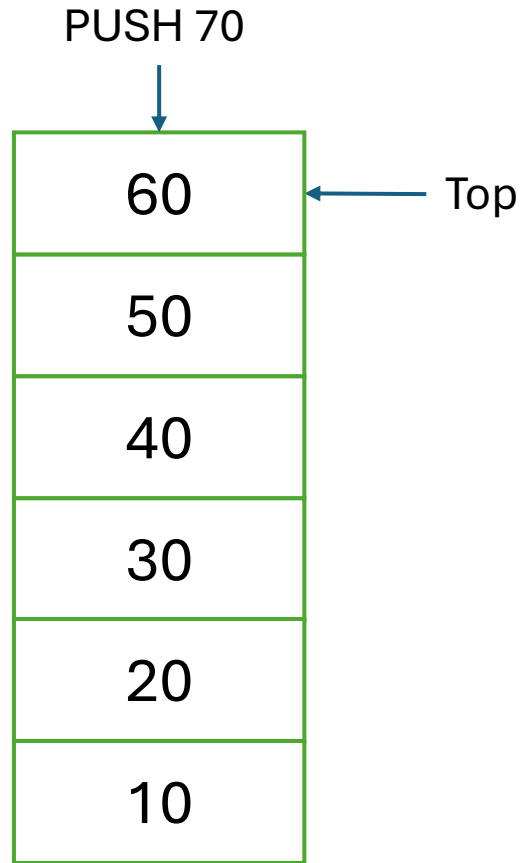
If there is no item stored, pop() has nothing to remove.

Why we check first:

isFull() helps prevent overflow, while isEmpty() helps prevent underflow.

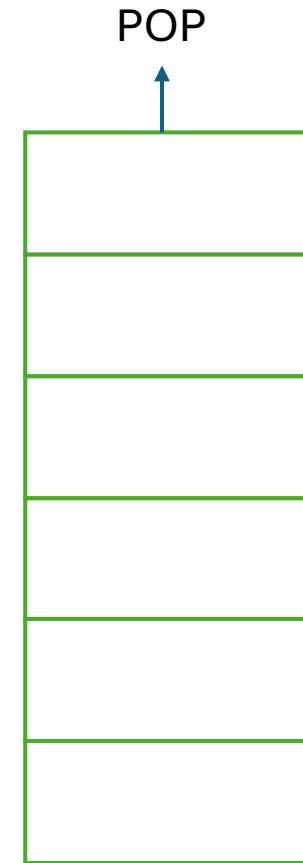
Overflow and Underflow Diagram

Stack Overflow

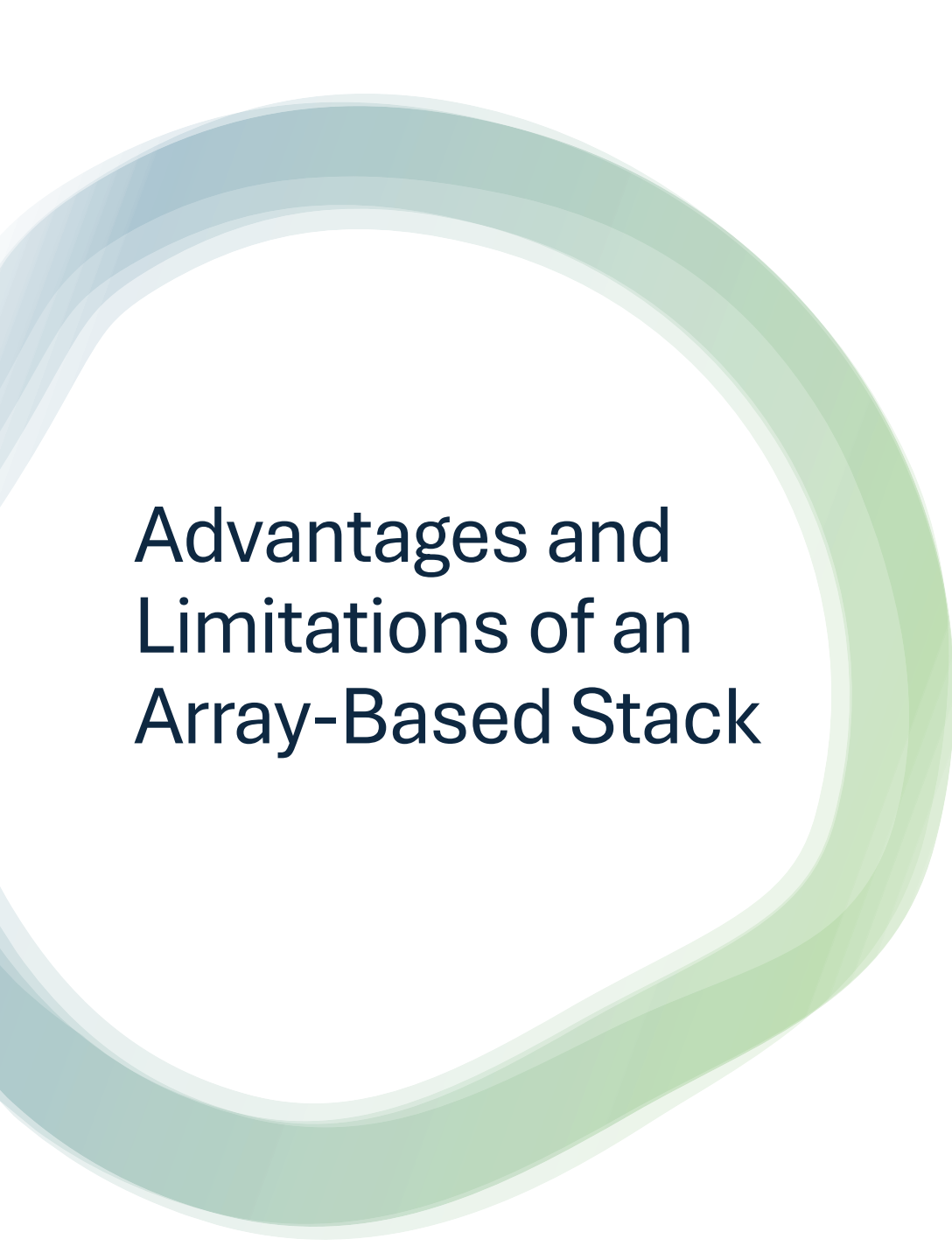


The stack is already full, so there is no empty space for 70. When we try to push 70 into the stack, stack overflow occurs.

Stack Underflow



The stack is empty, so there is no value to remove. When we try to use POP on an empty stack, stack underflow occurs.



Advantages and Limitations of an Array-Based Stack

▪ Advantages

- **Simple structure:** easy to understand and implement in C++.
- **Fast operations:** push, pop, and peek happen directly at the top.
- **Easy memory access:** array positions are straightforward to manage.
- **No complicated pointer handling** is required for this version.

▪ Limitations

- **Fixed size:** the capacity is decided in advance.
- If the Stack becomes full, another value cannot be inserted.
- Choosing a very large array can waste unused memory.
- The size cannot automatically grow like some dynamic data structures.

Real-Life and Computing Applications of Stack

➤ **Undo / Redo**

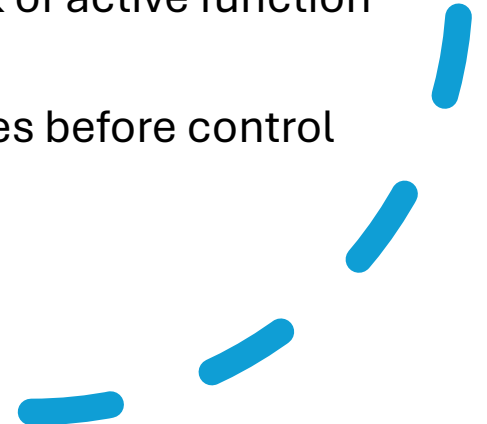
- When we type or edit something, the most recent action is normally the first action we want to undo.
- That matches LIFO because the latest action is stored on top.

➤ **Browser Navigation**

- Recently visited pages can be managed using Stack behaviour.
- When we go back, the most recently visited page is handled first.

➤ **Function Calls**

- Programs use Stack-like behaviour to keep track of active function calls.
- The most recently called function usually finishes before control returns to the earlier function.



Layperson Example: Stack of Cafeteria Trays

- I would use this because it is easy and different from only talking about numbers.
- Imagine a cafeteria where clean trays are placed one on top of another.
- Each new tray is placed on the **top** of the pile.
- When a person needs a tray, they normally take the **top tray first**.
- The tray that was placed last becomes the first one removed.
- **This is the same idea as LIFO in a Stack.**





From Theory to C++ Implementation

- **Now I will demonstrate the Stack using my C++ program.**
- During the live section I will show:
- how the Stack class is created,
- how the static array stores values,
- how top tracks the current element,
- how **push, pop, peek, isEmpty, isFull**, and **displayAll** work,
- and how the program handles overflow and underflow.
- After explaining the code, I will compile and run the program and test the operations using the menu.

References



Course Material – Dr. Victor Govindaswamy

I used the course material to understand what topics were required for the Stack section, especially LIFO, push, pop, peek, overflow, underflow, static-array implementation, and the diagram requirements.



GeeksforGeeks – www.geeksforgeeks.org

I reviewed Stack examples here to understand how push and pop change the top position and why overflow and underflow happen in an array-based Stack.



W3Schools – www.w3schools.com

I used W3Schools mainly as a C++ reference while working with classes, arrays, functions, loops, and conditions used in my Stack program.



Thank You

- Thank You for Watching!
- **Today I explained:**
- What a Stack is and how LIFO works
- Main Stack operations
- Static array implementation
- Push and Pop behavior
- Stack overflow and underflow
- Real-world applications
- Live C++ implementation and output
- I hope this video made the Stack data structure easier to understand.
Thank you for watching and learning with me.
- **Presented by: Akash Patel**
- **Instructor: Dr. Victor Govindaswamy**
- Like • Share • Subscribe
- Thank you for your time and attention.